Lab 10: Introduction to APIs: Wikipedia

INFO 4617/5617 · Week 10 · Chapter 10, Introduction to APIs: Wikipedia

Today we code together. Each step below says what we're building and why, and its code cell holds only the comments that frame it: we write the code live, in class. Nothing here is turned in. After most steps, a ⏱️ **Microsprint** gives you three minutes, alone or with a neighbor, to change one thing and bring a result back to the class. Every step ends with a 💬 **Discuss** box of questions to talk over before the next one; click it to open it.

By the end you'll have asked two of Wikipedia's APIs for one article's whole public record: its basic facts, every edit since 2003, its daily readers, who those readers are, and what the article connects to. Then you'll do the same with a client library in three lines.

The article: University of Colorado Boulder, the chapter's case. You'll choose your own article for the chapter's Recommended Exercises, due Sunday; this lab builds the tools they need.

Before class: open this notebook in the `webdata` environment (chapter 1), and have your email address ready for the User-Agent. Step 9 uses mwclient, which `webdata` doesn't include; Step 9 says how to install it.

On Friday, run every cell above where we stopped (**Run → Run All Above Selected Cell**).

> **Instructor key.** Every cell is filled in and was run against the live APIs on 2026-10-21. Every revision, pageview, and contribution request ends on or before 2026-09-30, so those outputs will match a rerun, unless Wikimedia reclassifies past traffic again (Step 7). Counts with no end date will drift: the article's and talk page's lengths (Steps 1–2), the categories, language editions, backlinks, and redirects (Step 8), and the libraries' versions. So will the `429: waiting` lines, which depend on who else shares your network address. The lab follows the textbook at commit d9306b7. Every cell ran live.

Step 0 — Set up (5 minutes)

Import `requests`, `pandas`, and `matplotlib`. Then define the User-Agent every request will send. Wikimedia's User-Agent policy asks every client to say who it is and how to reach its operator, in this form: `<client name>/<version> (<contact information>) <library name>/<version>`. Keep the first two parts in `CLIENT`, because Step 9's library adds its own name, and add `requests` and its version for `HEADERS`. A client the rate limits can identify only by its network address gets 10 requests a minute, shared with everyone on that network, which is everyone in this room. One with a User-Agent like this gets 200, and if it still gets a 429, the response's `Retry-After` header says how many seconds to wait.

```
# Import requests, pandas (as pd), and matplotlib.pyplot (as plt)
# Define CLIENT as "WebDataScience/1.0 (your-email@colorado.edu)", with your email
# Define HEADERS: a dict whose "User-Agent" is CLIENT, a space, then "python-requests/" and
 ↪  requests.__version__
# Define API, the Action API's address, and ARTICLE, the article's title
import requests
import pandas as pd
import matplotlib.pyplot as plt

CLIENT = "WebDataScience/1.0 (your-email@colorado.edu)"
HEADERS = {"User-Agent": f"{CLIENT} python-requests/{requests.__version__}"}
print(HEADERS)
API = "https://en.wikipedia.org/w/api.php"
ARTICLE = "University of Colorado Boulder"
```

```
{'User-Agent': 'WebDataScience/1.0 (your-email@colorado.edu) python-requests/2.34.2'}
```

💬 **Discuss**

- In week 2 we wrote a User-Agent too. What does this one add, and who reads it?
- Why does the rate limit treat everyone in this room as one client, unless each of us says who we are?
- Wikipedia's `robots.txt` (week 2) disallows `/w/`, yet the API lives at `/w/api.php`. Why isn't that a contradiction?

Step 1 — One request (5 minutes)

Every request to the MediaWiki Action API has four parts: the base URL, an `action` (what to do), parameters that narrow it, and a `format`. Build the parameters as a dictionary, and `requests` turns them into the URL. Ask for `prop=info`, the article's basic facts, and look at the status code before anything else.

```
# Build params: action=query, titles=ARTICLE, prop=info, format=json
# Send the request with HEADERS, print the status code, stop if it isn't 200, then print the JSON
params = {"action": "query", "titles": ARTICLE, "prop": "info", "format": "json"}
response = requests.get(API, params=params, headers=HEADERS, timeout=30)
print(response.status_code)
response.raise_for_status()
data = response.json()
print(data)
```

```
200
{'batchcomplete': '', 'query': {'pages': {'289672': {'pageid': 289672, 'ns': 0, 'title':
 ↪  'University of Colorado Boulder', 'contentmodel': 'wikitext', 'pagelanguage': 'en',
 ↪  'pagelanguagehtmlcode': 'en', 'pagelanguagedir': 'ltr', 'touched': '2026-10-18T22:35:28Z',
 ↪  'lastrevid': 1380813690, 'length': 95112}}}
```

💬 Discuss

- Our first request in week 1 fetched this same article as HTML. What's different about what came back this time?
- Print `response.url`. Where are the four parts of the request in it (week 5's URL anatomy), and which one would you change to ask about another article?
- What would a 403 or a 429 here tell you, and which of them is about you?

Step 2 — Read the JSON one level at a time (7 minutes)

The answer is nested: `data["query"]["pages"]`, then a key that is the page's ID. We don't know the ID in advance, so take the first (and only) value. Print the title, the page ID, and the article's length in bytes.

```
# Go down one level at a time: data -> "query" -> "pages" -> the one page inside
# Print its title, pageid, and length
pages = data["query"]["pages"]
page = list(pages.values())[0]
print(page["title"], page["pageid"], f"{page['length']:,} bytes")
```

```
University of Colorado Boulder 289672 95,112 bytes
```

⏱️ Microsprint (3 minutes, alone or in pairs)

Do: ask `prop=info` about three titles in one request, joined with `|`: the article, its talk page (`Talk:` and the title), and a misspelling of the title. Print each page's key, title, and length. This is the chapter's tour of the talk page, and its warning about missing pages in "Common Issues to Debug".

Share: the talk page's length, and what the API did with the misspelling.

```
# Microsprint: one request about three titles, joined with "|": ARTICLE, "Talk:" + ARTICLE, and a
 ↪  misspelling
# Send it as in Step 1; then for each key and page under ["query"]["pages"], print the key, the
 ↪  title,
#   and the length, or "missing" when there isn't one
titles = [ARTICLE, f"Talk:{ARTICLE}", "University of Colorado Bolder"]
three = {"action": "query", "titles": "|".join(titles), "prop": "info", "format": "json"}
response = requests.get(API, params=three, headers=HEADERS, timeout=30)
response.raise_for_status()
for key, info in response.json()["query"]["pages"].items():
    print(key, info["title"], info.get("length", "missing"))
```

```
-1 University of Colorado Bolder missing
289672 University of Colorado Boulder 95112
2634293 Talk:University of Colorado Boulder 682
```

💬 Discuss

- Why is the page ID a key rather than a value? What key did the misspelled title get, and what would `page["pageid"]` do with it?
- In week 6 we scraped Wikipedia infoboxes with BeautifulSoup. Which approach breaks first when Wikipedia redesigns its pages?
- The talk page is short, though editors have argued on it for years. Where did its discussions go, and what would you need to know before comparing two pages by length?

Step 3 — Every edit, one batch at a time (15 minutes)

`prop=revisions` returns an article's edits, newest first, at most 500 per request. When there are more, the response carries a `"continue"` key: add its contents to your parameters and ask again, until it's gone. Forgetting this loop is the most common API bug there is, because the first batch looks like a complete answer.

This step sends many requests, so first write `get(url, params)`, the one way every request from here on goes out. It always sends `HEADERS`, and when the answer is a 429 it waits as many seconds as `Retry-After` says and asks again, up to three times, as the rate-limit page asks.

Then write `get_revisions(title)`. Use `rvstart` to fix the newest edit you'll accept, `2026-09-30T23:59:59Z`, so that running it next week gives today's answer. Call `raise_for_status()` before `.json()`: if the rate limit still answers, you'll see why instead of a confusing JSON error.

```
# Import time. Define get(url, params=None):
#   up to four tries: request url with params and HEADERS (timeout=30); return the response
#   unless it is a 429, in which case time.sleep() for int(response.headers["Retry-After"])
 ↪  seconds
# Define get_revisions(title):
#   params: action=query, titles, prop=revisions, rvprop=ids|timestamp|user|size|comment,
#           rvlimit=max, rvstart="2026-09-30T23:59:59Z", format=json
#   loop: get(API, params), raise_for_status, keep this batch's revisions,
#         and if "continue" is in the response, add it to params; otherwise stop
#   return the revisions as a DataFrame
# Call it for ARTICLE and print how many revisions came back
import time

def get(url, params=None):
    for attempt in range(4):
        response = requests.get(url, params=params, headers=HEADERS, timeout=30)
        if response.status_code != 429 or attempt == 3:
            return response
```

```
        wait = int(response.headers.get("Retry-After", 5))
        print(f"429: waiting {wait} seconds")
        time.sleep(wait)

def get_revisions(title, until="2026-09-30T23:59:59Z"):
    params = {"action": "query", "titles": title, "prop": "revisions",
              "rvprop": "ids|timestamp|user|size|comment",
              "rvlimit": "max", "rvstart": until, "format": "json"}
    revisions = []
    while True:
        response = get(API, params)
        response.raise_for_status()
        data = response.json()
        page = list(data["query"]["pages"].values())[0]
        revisions.extend(page.get("revisions", []))
        if "continue" in data:
            params.update(data["continue"])
        else:
            break
    return pd.DataFrame(revisions)

revisions = get_revisions(ARTICLE)
print(f"{len(revisions):,} revisions")
```

```
429: waiting 43 seconds
```

```
2,988 revisions
```

💬 Discuss

- How many requests did `get_revisions()` send? Without the `continue` loop, how many revisions would we have, and would anything warn us?
- Why fix `rvstart` at a date in the past? What does that give someone who reruns your notebook next year?
- Week 7's Wayback captures and this history are both versions of a page over time. Which is the more complete record, and why?

Step 4 — Chart the edits, and find the editors (7 minutes)

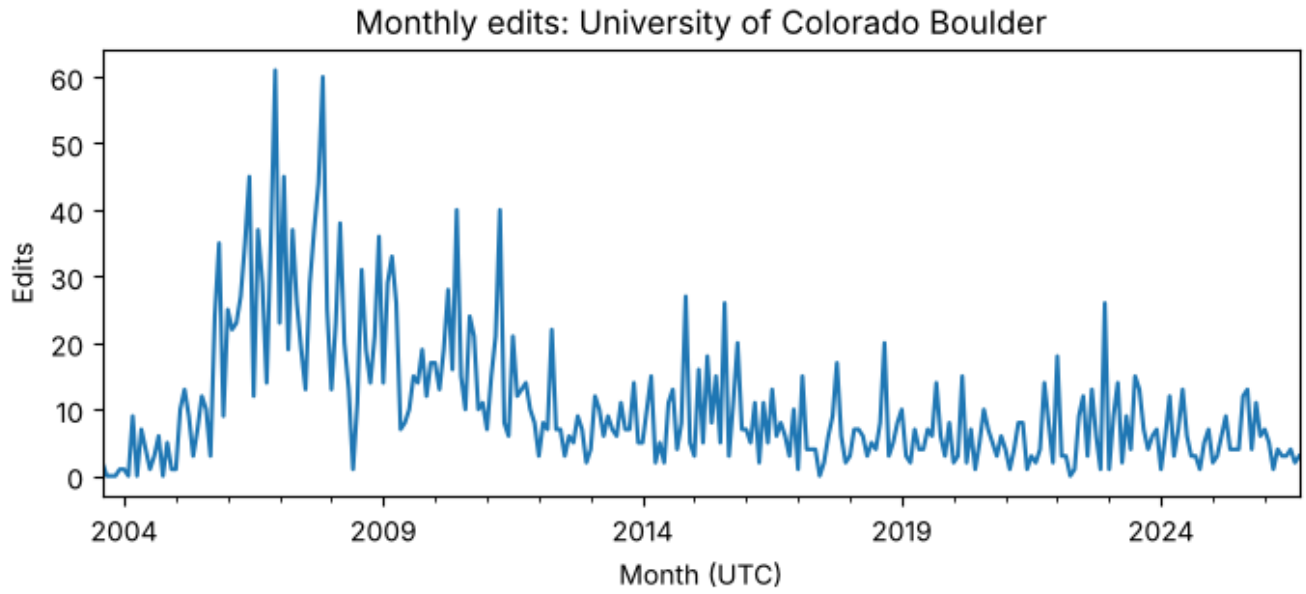
Turn `timestamp` into dates, count edits per month with `resample("ME")`, and plot them, with a title and a label on each axis. Then use `value_counts()` on `user` to see who edits most, and what share of all edits the top ten made. The chapter's ”Analyzing Revision History” does the same.

```
# Convert timestamp to datetimes, count edits per month, and plot them with a title and axis
↪  labels
# Show the ten most active editors, and the share of all edits they made
revisions["timestamp"] = pd.to_datetime(revisions["timestamp"])
monthly = revisions.set_index("timestamp").resample("ME").size()

fig, ax = plt.subplots(figsize=(8, 3))
monthly.plot(ax=ax)
ax.set_title(f"Monthly edits: {ARTICLE}")
ax.set_xlabel("Month (UTC)")
ax.set_ylabel("Edits")
plt.show()

top = revisions["user"].value_counts().head(10)
```

```
print(top)
print(f"Top 10 share: {top.sum() / len(revisions):.1%}")
```




```
user
Kerowyn              92
MECU                 82
ElKevbo              78
Contributor321       49
Kvng                 40
Wordbuilder          37
Fleminra             27
Levineps             24
Corkythehornetfan    23
Rookkey              22
Name: count, dtype: int64
Top 10 share: 15.9%
```

⏱️ Microsprint (3 minutes, alone or in pairs)

Do: count the edits made without an account. The API marks an edit from an IP address with a value in the `anon` column, and one from a temporary account with a value in `temp`; `notna()` finds them. What share of all edits are they, and what share of the edits since 2021? Count them, and don't print them: an IP address can point to a household. A short version of Additional Exercise 6's question of whether anonymous editors count.

Share: the two shares.

```
# Microsprint: an edit without an account has a value in "anon" (an IP address) or "temp" (a
 ↪  temporary account)
# Count them without printing them: what share of all edits, and of the edits since 2021?
no_account = revisions["anon"].notna() | revisions["temp"].notna()
print(f"All {len(revisions):,} edits: {no_account.mean():.1%} without an account")
recent = revisions["timestamp"].dt.year >= 2021
print(f"The {recent.sum()} edits since 2021: {no_account[recent].mean():.1%} without an account")
```

```
All 2,988 edits: 35.2% without an account
The 422 edits since 2021: 25.6% without an account
```

💬 **Discuss**

- Most edits came in the article's first few years. What does that suggest about how an article matures?
- In the microsprint, many edits came without an account, and each IP address counts as its own editor. What does that do to a count of editors, or to the share the top ten made?
- Usernames and IP addresses are both public here. By week 2's privacy question, is it fine to publish a ranking of an article's editors, or a list of its IP addresses?

Step 5 — A second API for readers (5 minutes)

Pageviews come from a different API, Wikimedia's REST API, whose parameters are parts of the URL's path rather than a dictionary: project, access, **agent** (who is reading), article, granularity, start, and end. Define `get_pageviews()` in this cell, and use it in the next. Titles in the path use underscores, not spaces.

```
# Define get_pageviews(title, start, end, agent="user", granularity="daily"):
#   build the URL: https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article
#                  /en.wikipedia/all-access/{agent}/{article}/{granularity}/{start}/{end}
#   request it with get(), raise_for_status, make a DataFrame from ["items"]
#   turn "timestamp" (YYYYMMDDHH) into dates, and return the timestamp and views columns
def get_pageviews(title, start, end, agent="user", granularity="daily"):
    article = title.replace(" ", "_")
    url = ("https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article"
           f"/en.wikipedia/all-access/{agent}/{article}/{granularity}/{start}/{end}")
    response = get(url)
    response.raise_for_status()
    views = pd.DataFrame(response.json()["items"])
    views["timestamp"] = pd.to_datetime(views["timestamp"], format="%Y%m%d%H")
    return views[["timestamp", "views"]]
```

💬 **Discuss**

- Week 5 called this a path-segment API, and the Action API a query-parameter one. Why would a title with a slash, such as AC/DC, break `get_pageviews()` but not Step 1's request?
- Why doesn't this function need a `continue` loop?
- In week 1 we asked this API for `all-agents`. What changes when we ask for `user`?

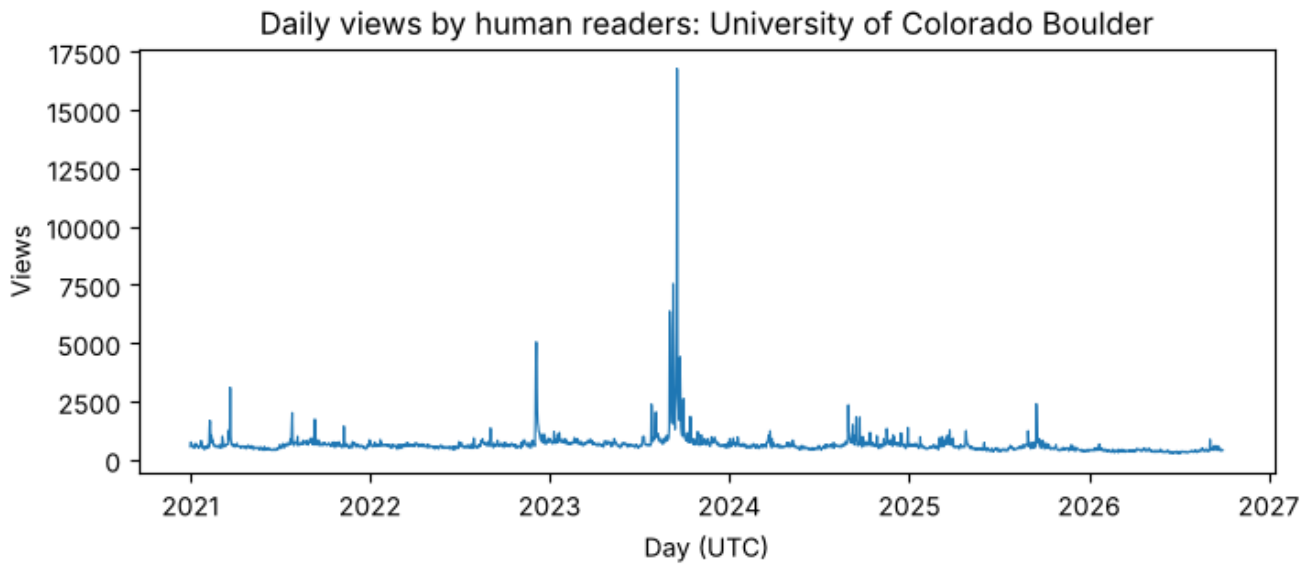
Step 6 — Plot the readers, and find the biggest day (8 minutes)

Get daily views from January 1, 2021 to September 30, 2026, plot them with a title and axis labels, and find the peak with `idxmax()`. What happened that day? The API's days are UTC days, so an evening event in Colorado lands on the next one.

```
# Get daily views for ARTICLE from 20210101 to 20260930, and plot them with a title and axis
 ↪  labels
# Find the day with the most views, and print its date and count
views = get_pageviews(ARTICLE, "20210101", "20260930")

fig, ax = plt.subplots(figsize=(8, 3))
ax.plot(views["timestamp"], views["views"], linewidth=0.8)
ax.set_title(f"Daily views by human readers: {ARTICLE}")
ax.set_xlabel("Day (UTC)")
ax.set_ylabel("Views")
plt.show()

peak = views.loc[views["views"].idxmax()]
print(peak["timestamp"].date(), f"{peak['views']:,} views")
```




```
2023-09-17 16,767 views
```

💬 Discuss

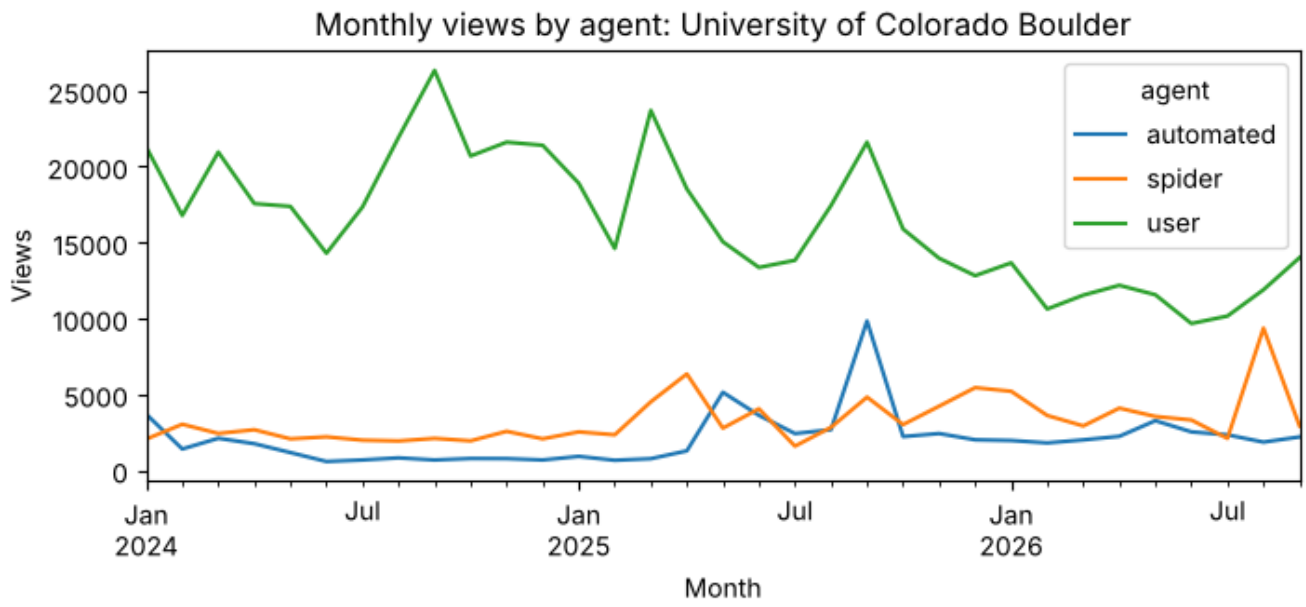
- The API's days are UTC days. What mistake could you make reporting the peak as a Colorado date?
- What happened around the peak? How would you check it with data, such as the revisions around that date, rather than from memory?
- A spike measures attention, not approval. Which claims can it support, and which can't it?

Step 7 — Who is reading? (9 minutes)

A pageview is not a person. Wikimedia's definition splits its counts by agent: `user` (people), `spider` (crawlers that say what they are), and `automated` (bots that don't). Call `get_pageviews()` once for each, with monthly granularity, for 2024 to September 2026, and plot the three together. When Wikimedia improved its bot detection in 2025, it moved past traffic from one agent to another. The chapter's "What a Pageview Counts" has the history.

```
# For each agent in user, spider, automated: get monthly views from 20240101 to 20260930
# Put them in one DataFrame, one column per agent (pivot), and plot it with a title and axis
 ↪  labels
frames = []
for agent in ["user", "spider", "automated"]:
    monthly_views = get_pageviews(ARTICLE, "20240101", "20260930", agent=agent,
 ↪  granularity="monthly")
    monthly_views["agent"] = agent
    frames.append(monthly_views)
by_agent = pd.concat(frames).pivot(index="timestamp", columns="agent", values="views")

ax = by_agent.plot(figsize=(8, 3), title=f"Monthly views by agent: {ARTICLE}")
ax.set_xlabel("Month")
ax.set_ylabel("Views")
plt.show()
```



⏱️ Microsprint (3 minutes, alone or in pairs)

Do: check the step's three lines. Ask for `agent="all-agents"`, monthly, for the same months, and check that each month is the sum of `user`, `spider`, and `automated`. Then divide `user` by it, and find the month when people were the smallest share of the views.

Share: whether the sum checks out, and that month and its share.

```
# Microsprint: get monthly all-agents views for the same months; check each equals user + spider
 ↪  + automated
# Then divide user by all-agents, and print the month when people were the smallest share, with
 ↪  the share
all_agents = get_pageviews(ARTICLE, "20240101", "20260930", agent="all-agents",
 ↪  granularity="monthly")
total = all_agents.set_index("timestamp")["views"]
print("Sum checks out:", (by_agent.sum(axis=1) == total).all())
user_share = by_agent["user"] / total
print(f"{user_share.idxmin():%B %Y}: {user_share.min():.0%} of views were people")
```

```
Sum checks out: True
August 2026: 51% of views were people
```

💬 Discuss

- The microsprint found the month when people were the smallest share of views. How would a study that used `all-agents` have read that month?
- Wikimedia moved past traffic from one agent to another when its bot detection improved. What does that mean for anyone who downloaded counts before then?
- Our Selenium visits in week 8 came from a real browser. Which agent should they count as, and which do you think they did?

Step 8 — One function for everything else (12 minutes)

Links, categories, templates, languages, and what links here are all `action=query`, with a different `prop` or `list`, and they all paginate the same way. So write the loop once: `query_all(params)` follows `"continue"` and returns every batch. Then count the article's categories, its language editions, and the articles that link to it.

Look at the first few categories: some are **hidden** maintenance categories that readers never see. Add `clshow="!hidden"` to leave them out.

```
# Define query_all(params): add action=query and format=json, then loop like Step 3,
#   collecting each response's "query" part; return the list of batches
# Use it to count: categories (prop=categories, cllimit=max), with and without clshow="!hidden";
#   language editions (prop=langlinks, lllimit=max);
#   articles linking here (list=backlinks, bltitle, bllimit=max, blnamespace=0)
def query_all(params):
    params = {"action": "query", "format": "json", **params}
    batches = []
    while True:
        response = get(API, params)
        response.raise_for_status()
        data = response.json()
        batches.append(data["query"])
        if "continue" in data:
            params.update(data["continue"])
        else:
            break
    return batches

def page_items(title, prop, extra):
    items = []
    for batch in query_all({"titles": title, "prop": prop, **extra}):
        for page in batch["pages"].values():
            items.extend(page.get(prop, []))
    return items

categories = page_items(ARTICLE, "categories", {"cllimit": "max"})
visible = page_items(ARTICLE, "categories", {"cllimit": "max", "clshow": "!hidden"})
languages = page_items(ARTICLE, "langlinks", {"lllimit": "max"})
backlinks = [b for batch in query_all({"list": "backlinks", "bltitle": ARTICLE,
                                       "bllimit": "max", "blnamespace": 0})
             for b in batch["backlinks"]]
print(f"{len(categories)} categories, {len(visible)} visible to readers")
print(f"{len(languages)} other language editions")
print(f"{len(backlinks):,} articles link here")
```

```
429: waiting 1 seconds
```

```
429: waiting 49 seconds
```

```
40 categories, 9 visible to readers
38 other language editions
3,297 articles link here
```

⏱️ Microsprint (3 minutes, alone or in pairs)

Do: find which other titles send readers to this article. `prop=redirects` isn't in the chapter: read its documentation for the name of its limit parameter, and add `rdnamespace=0` for articles only. Use `page_items()`, and print how many there are and the first eight. A short version of Additional Exercise 4.

Share: how many there are, and the most surprising one.

```
# Microsprint: which titles redirect here? Use page_items() with prop="redirects",
#   its limit parameter (from the documentation) set to "max", and rdnamespace=0
# Print how many there are, and the first eight titles
```

```
redirects = page_items(ARTICLE, "redirects", {"rdlimit": "max", "rdnamespace": 0})
print(f"{len(redirects)} titles redirect here")
print([r["title"] for r in redirects[:8]])
```

```
70 titles redirect here
['University of Colorado, at Boulder', 'University of Colorado, Boulder', 'University Memorial
 ↪  Center at the University of Colorado at Boulder', 'Tocqueville Initiative', 'University of
 ↪  colorado at boulder', 'University of Colorado in Boulder', 'University of Colorado -
 ↪  Boulder', 'Western Academic Talent Search']
```

💬 **Discuss**

- Why did `query_all()` work for categories, language links, backlinks, and the microsprint's redirects without a change?
- Hidden categories are maintenance labels that readers never see. Should a study of how Wikipedia classifies a topic include them?
- Dozens of other editions have an article on this topic. What could you learn by comparing them, as week 6 did with French infoboxes?

Step 9 — The same with a client library (7 minutes)

mwclient does the continuation for you. If you haven't installed it, run `%conda install -y -c conda-forge mwclient` in a new cell first; conda-forge has version 0.10.1. Ask it for the same article's categories, language editions, and revisions. In mwclient 0.10, `limit` is how many results to ask for in each request; version 0.11 calls it `api_chunk_size`.

Two things stay raw. mwclient speaks only the Action API, so pageviews still need Step 5. And it doesn't wait on a 429: it stops with an error. So make the `Site` with `do_init=False`, which sends nothing yet, and give its session (`site.connection`) a retry rule from `urllib3`, the library under `requests`: on a 429, wait as long as `Retry-After` says and try again, up to five times. Then `site.site_init()` sends the first request.

```
# Import mwclient and print its version
# Make a Site for en.wikipedia.org with clients_useragent=CLIENT (mwclient adds its own name and
 ↪  version)
#   and do_init=False; mount HTTPAdapter(max_retries=Retry(total=5, status_forcelist=[429]))
#   on site.connection for "https://", then call site.site_init()
# Get the page, then count its categories, its language editions,
#   and its revisions up to 2026-09-30T23:59:59Z, 500 per request (limit=500)
import mwclient
print(mwclient.__version__)

from requests.adapters import HTTPAdapter
from urllib3.util import Retry

site = mwclient.Site("en.wikipedia.org", clients_useragent=CLIENT, do_init=False)
site.connection.mount("https://", HTTPAdapter(max_retries=Retry(total=5,
 ↪  status_forcelist=[429])))
site.site_init()
print(site.connection.headers["User-Agent"])
page = site.pages[ARTICLE]
print(len(list(page.categories())), "categories")
print(len(list(page.langlinks())), "language editions")
print(len(list(page.revisions(start="2026-09-30T23:59:59Z", limit=500))), "revisions")
```

```
0.10.1
```

```
WebDataScience/1.0 (your-email@colorado.edu) mwclient/0.10.1
 ↪  (https://github.com/mwclient/mwclient)

40 categories
38 language editions

2988 revisions
```

⏱️ Microsprint (3 minutes, alone or in pairs)

Do: ask mwclient for the edits that the chapter's "Getting User Contributions" got with `list=usercontribs`: Jimbo Wales's, from January to September 2026. Use `site.usercontributions()`, whose `start` is the later date, because it goes newest first. Count the edits, then count them by `ns`, the namespace: 0 is articles, 1 their talk pages, 3 user talk pages. This is the last part of Additional Exercise 4: the same request with mwclient.

Share: the total (does it match the chapter's?), and how many were edits to articles.

```
# Microsprint: list site.usercontributions("Jimbo Wales", start="2026-09-30T23:59:59Z",
#   end="2026-01-01T00:00:00Z", limit=500) into a DataFrame
# Print how many edits, and count them by "ns" with value_counts()
contribs = pd.DataFrame(list(site.usercontributions("Jimbo Wales", start="2026-09-30T23:59:59Z",
                                                    end="2026-01-01T00:00:00Z", limit=500)))
print(f"{len(contribs)} edits")
print(contribs["ns"].value_counts())
```

```
351 edits
ns
3    243
1     79
0     12
4     10
2      4
5      3
Name: count, dtype: int64
```

💬 Discuss

- What did mwclient do for you that you wrote by hand in Steps 3 and 8? What didn't it do?
- Most of those edits were on talk pages, not articles. What would a study that counted "edits" without splitting them by namespace get wrong?
- Week 3 was about APIs that closed. Wikipedia's has stayed open. What keeps it open, and what would your project lose if it closed?

Step 10 — What did we learn? (5 minutes)

In a few sentences, as comments:

- Steps 3 and 9 counted the revisions two ways. Did they agree, and what would it mean if they didn't?
- In Step 7 and its microsprint, how much of the article's traffic was people? What would a study that used `all-agents` have gotten wrong?
- Which of today's numbers would be different if you ran this notebook next year, and which wouldn't? Why?
- What did mwclient do for you in Step 9 and its microsprint, and what did you still do yourself?

Recommended Exercises ahead (in the chapter's companion notebook, due Sunday):

- Step 1, choose an article and get its pageviews: Steps 5–6 built and used `get_pageviews()`, with a window that ends before today.

- Step 2, find the spike: Step 6's `idxmax()` and its warning about UTC days; Step 3's revisions are where to look around the date.
- Step 3, get the revision history: Step 3's `get()` and `get_revisions()`, with the `continue` loop.
- Step 4, profile the editors: Step 4's `value_counts()`, and its microsprint on edits without an account.
- Step 5, align edits with attention: Step 4's `resample("ME")`, and Step 7's monthly granularity.
- Step 6, map the neighborhood: Step 8's `query_all()` and `page_items()`, which the chapter calls `page_list()`; wikilinks are `prop=links`, with `pllimit`.
- Step 7, interpret: the Discuss questions after Steps 4, 6, and 7.

```
# Your answers here, as comments
# Steps 3 and 9 agree: 2,988 revisions up to 2026-09-30, from our loop and from mwclient's.
#   If they didn't, one had stopped early, most likely at a missed "continue" or an unhandled
 ↪  429.
# Most, but not all: in August 2026 only 51% of the article's views were `user` (Step 7's
 ↪  microsprint),
#   because crawlers spiked that month. all-agents would have read that crawler spike as readers,
#   and by a different amount each month.
# The revisions (2,988) and the pageviews (peak 16,767 on 2023-09-17) end at fixed dates, so
 ↪  they'd stay,
#   unless Wikimedia reclassifies past traffic again. Nothing fixes the length (95,112 bytes),
 ↪  the backlinks
#   (3,297), the redirects (70), the categories, or the editions: the chapter printed 95,273
 ↪  bytes and 3,294 backlinks.
# mwclient followed "continue" for us: 40 categories, 38 editions, 2,988 revisions, and Jimbo
 ↪  Wales's
#   351 edits (the chapter's count) in a line each. We still gave it a retry rule for 429s, and
 ↪  pageviews
#   still need Step 5's get_pageviews().
```

Friday: from the lab to the textbook (15 minutes)

The last 15 minutes go to the textbook. Look back at what the lab turned up in chapter 10:

- **An output that has changed.** The chapter printed its outputs in October 2026. Compare a few with yours, such as the article's length (Step 2) and the number of articles that link to it (Step 8). Which of them should the chapter fix to a date, and how?
- **A cell that broke.** Did a 429 stop one of your cells? The chapter's functions send `requests.get()` directly, and its mwclient code has no retry rule. Where should the chapter show how to wait?
- **A sentence that confused.** "Common Issues to Debug" says that a missing page's ID will be negative. What did Step 2's microsprint show?
- **A microsprint that would make a good exercise.** Checking that `all-agents` is the sum of the other three, or counting the edits made without an account, could be an Additional Exercise.

The revision framework says how to propose a change, and what each kind needs to be accepted.